

Diseno y desarrollo de una arquitectura multiagente
basada en LangGraph
para procesamiento de datos y deteccion de
anomalias industriales

Autor: pendiente

Curso academico: pendiente

Resumen

Este Trabajo Fin de Master propone una arquitectura multiagente para el procesamiento, modelado y detección de anomalías en entornos industriales. El caso de uso inicial se centra en rodamientos de motores eléctricos monitorizados mediante señales de vibración. La solución combina agentes coordinados mediante LangGraph con ejecutores Python deterministas, de forma que las decisiones se representen mediante contratos estructurados y las transformaciones sobre datos sean reproducibles, auditables y testeables.

Abstract

This Master's Thesis proposes a multi-agent architecture for data processing, modeling and anomaly detection in industrial environments. The initial use case focuses on electric motor bearings monitored through vibration signals. The solution combines agents coordinated with LangGraph and deterministic Python executors, so that decisions are represented through structured contracts and data transformations remain reproducible, auditable and testable.

Índice general

1. Introduccion	1
1.1. Contexto	1
1.2. Motivacion	1
1.3. Problema	1
1.4. Objetivo general	2
1.5. Objetivos especificos	2
1.6. Alcance y limitaciones iniciales	2
2. Estado del arte	3
2.1. Mantenimiento predictivo y deteccion de anomalias	3
2.2. Modelos de deteccion de anomalias	3
2.3. Sistemas multiagente aplicados a pipelines de datos	3
2.4. Datasets publicos de referencia	4
3. Arquitectura propuesta	5
3.1. Principios de diseno	5
3.2. Separacion entre agentes y ejecutores	5
3.3. Nodos previstos	5
3.4. Estado global	6
3.5. Contratos estructurados	6
3.6. Trazabilidad	7
4. Metodologia	8
4.1. Caso de uso industrial	8
4.2. Pipeline de datos	8
4.3. Manifiesto del dataset	8
4.4. Extraccion y perfilado	9
4.5. Estructuracion temporal	9
4.6. Particiones	9
4.7. Metricas	9
5. Implementacion	10
5.1. Estructura del repositorio	10
5.2. Estado actual	10
5.3. Estado global implementado	10
5.4. Contratos Pydantic implementados	11
5.5. Artefactos previstos	11
5.6. Ejecutor de manifiesto	11
5.7. Dependencias	12

5.8. Verificacion actual	12
5.9. Siguiente paso de implementacion	12
6. Experimentos	13
6.1. Protocolo experimental	13
6.2. Experimento base	13
6.3. Experimentos posteriores	13
7. Resultados	14
7.1. Resultados cuantitativos	14
7.2. Analisis cualitativo	14
7.3. Limitaciones observadas	14
8. Conclusiones	15
8.1. Conclusiones principales	15
8.2. Trabajo futuro	15

Índice de figuras

Índice de tablas

7.1. Plantilla de resultados para modelos de deteccion de anomalias.	14
--	----

Capítulo 1

Introduccion

1.1. Contexto

La deteccion de anomalias en maquinaria industrial es una tarea relevante para mejorar la fiabilidad, reducir paradas no planificadas y apoyar estrategias de mantenimiento predictivo. En sistemas rotativos, como motores electricos, turbinas, bombas o rodamientos, las senales de vibracion permiten observar cambios de comportamiento asociados a desgaste, desbalanceo, defectos mecanicos o degradacion progresiva.

Este trabajo se centra inicialmente en rodamientos de motores electricos, usando como dataset principal CWRU Bearing Dataset [2]. El dataset NASA IMS Bearing se conserva como fuente futura para validar escenarios de degradacion run-to-failure [4].

1.2. Motivacion

El interes del proyecto no se limita a entrenar un modelo de deteccion de anomalias. El objetivo es construir una arquitectura capaz de coordinar el flujo completo de datos: ingestion, perfilado, limpieza, estructuracion temporal, entrenamiento, evaluacion, auditoria e informe tecnico. Para evitar transformaciones opacas o no reproducibles, los agentes proponen decisiones estructuradas y los ejecutores deterministas aplican las operaciones reales.

1.3. Problema

El problema abordado es la deteccion automatica de patrones anomalos en senales de vibracion procedentes de rodamientos. En el MVP se formulara como una tarea binaria:

- clase normal: operacion sin fallo;
- clase anomala: cualquier fallo de rodamiento disponible en el dataset.

La clasificacion por tipo de fallo se mantiene como extension posterior. Esta decision reduce el alcance inicial y permite validar primero la trazabilidad del pipeline multiagente.

1.4. Objetivo general

Disenar e implementar una arquitectura multiagente basada en LangGraph para procesar datos industriales de vibracion y detectar anomalias de forma reproducible, auditable y extensible.

1.5. Objetivos especificos

1. Definir una estructura de proyecto que separe codigo, memoria academica y recursos.
2. Seleccionar un caso de uso industrial y datasets publicos adecuados.
3. Disenar un pipeline de datos reproducible para senales de vibracion.
4. Definir contratos estructurados para agentes y ejecutores.
5. Implementar ejecutores deterministas para perfilado, limpieza, estructuracion temporal, modelado y evaluacion.
6. Construir un MVP local antes de abordar Docker, SLURM o frontend.
7. Generar informes tecnicos trazables a partir de metricas y artefactos.

1.6. Alcance y limitaciones iniciales

El alcance inicial queda limitado a ejecucion local, ficheros crudos en disco y un dataset principal. No se abordaran en el MVP la ejecucion distribuida, streaming industrial, integracion SLURM, frontend ni persistencia avanzada. NASA IMS se mantiene como extension experimental una vez validado el flujo sobre CWRU.

Capítulo 2

Estado del arte

2.1. Mantenimiento predictivo y deteccion de anomalias

Esta seccion desarrollara el contexto de mantenimiento predictivo, monitorizacion de condicion y deteccion de anomalias en maquinaria rotativa.

Aspectos a cubrir:

- diferencias entre mantenimiento correctivo, preventivo y predictivo;
- papel de las senales de vibracion en rodamientos y motores;
- fallos comunes: pista interior, pista exterior, bola y desbalanceo;
- diferencias entre deteccion de anomalias, diagnostico y pronostico.

2.2. Modelos de deteccion de anomalias

El MVP comparara primero modelos clasicos y reproducibles:

- Isolation Forest;
- One-Class SVM;
- Local Outlier Factor;
- PCA con error de reconstruccion.

Los autoencoders densos y modelos recurrentes se consideraran en fases posteriores, cuando el pipeline de tensores este validado.

2.3. Sistemas multiagente aplicados a pipelines de datos

Esta seccion describira el uso de arquitecturas multiagente para dividir responsabilidades entre nodos especializados. El enfoque del proyecto separa:

- decisiones de alto nivel tomadas por agentes;
- validacion de contratos mediante esquemas estrictos;
- ejecucion determinista de transformaciones sobre datos;
- auditoria de rutas, configuraciones, metricas y artefactos.

2.4. Datasets publicos de referencia

El dataset principal sera CWRU Bearing Dataset [2, 3, 1]. NASA IMS Bearing Dataset se reserva como extension para escenarios de degradacion temporal [4, 5].

Capítulo 3

Arquitectura propuesta

3.1. Principios de diseno

La arquitectura se basa en cuatro principios:

1. separar codigo, memoria academica y recursos;
2. mantener los datos crudos inmutables;
3. representar decisiones mediante contratos estructurados;
4. ejecutar transformaciones mediante modulos Python deterministas.

3.2. Separacion entre agentes y ejecutores

Los agentes no escriben ni ejecutan codigo arbitrario sobre los datos. Su papel es proponer configuraciones, interpretar perfiles y decidir la siguiente fase del flujo. Los ejecutores leen artefactos desde disco, aplican operaciones deterministas y devuelven rutas, metricas y logs.

3.3. Nodos previstos

El grafo minimo previsto contiene:

- supervisor;
- agente limpiador;
- ejecutor de limpieza;
- agente estructurador;
- ejecutor de estructuracion;
- agente modelador;
- ejecutor de modelado;
- evaluador;

- redactor;
- revision humana para trabajos costosos.

3.4. Estado global

El estado global de LangGraph no almacenara datasets completos. Debe contener rutas, resúmenes y artefactos:

- identificador de ejecución;
- contexto del proyecto;
- rutas de datos crudos, perfiles, datos limpios, tensores e informes;
- configuraciones de limpieza, estructuración y modelado;
- métricas y evaluación;
- errores, aprobaciones humanas y artefactos generados.

En la implementación inicial se ha definido un modelo canónico ‘TFMStateModel’ validado con Pydantic y una vista ‘TFMState’ basada en ‘TypedDict’ para su uso en LangGraph. Esta separación permite validar contratos de forma estricta antes de entregar el estado al grafo y, al mismo tiempo, mantener una representación ligera y serializable.

El estado inicial del caso CWRU queda fijado en la fase ‘dataset_manifest’, con el nodo siguiente ‘manifest_executor’. El directorio crudo se registra como `codigo/data/raw/cwru_bearing/mat`, el canal principal es ‘DE_time’ y la frecuencia objetivo es 12 kHz. Las rutas de artefactos posteriores permanecen como valores nulos hasta que los ejecutores correspondientes las generen.

3.5. Contratos estructurados

Los contratos iniciales incluyen:

- ‘ProjectContext’, que define dominio, máquina, dataset, objetivo, canal principal y frecuencia objetivo;
- ‘DatasetManifestRow’ y ‘DatasetManifest’, que formalizan el inventario de archivos crudos y sus etiquetas;
- ‘DatasetProfileSummary’, que resume el perfil sin almacenar señales;
- ‘CleaningConfig’, ‘StructuringConfig’ y ‘ModelingConfig’, que fijan las configuraciones que podrán proponer los agentes;
- ‘SupervisorDecision’, ‘CleaningDecision’, ‘StructuringDecision’, ‘ModelingDecision’, ‘EvaluationDecision’ y ‘ReportDecision’, que estructuran las respuestas de los agentes;

- ‘ExecutorResult’ y resultados especializados, que normalizan las salidas de los ejecutores deterministas;
- ‘MetricsReport’ y ‘EvaluationResult’, que estructuran la evaluacion;
- ‘PipelineError’, ‘HumanApproval’ y ‘ArtifactRef’, que mantienen trazabilidad de errores, revisiones y artefactos.

Estos esquemas aplican validacion estricta y rechazan campos no declarados. Por ejemplo, una fila normal del manifiesto debe declarar los campos de fallo como nulos, mientras que una fila con fallo debe incluir tipo de fallo y diametro. De forma analoga, un resultado de ejecutor con estado fallido debe incluir un error estructurado, y una decision no terminal del supervisor debe indicar el siguiente nodo del grafo.

3.6. Trazabilidad

Cada ejecucion debe permitir responder:

- que datos se usaron;
- que configuracion fue validada;
- que ejecutor produjo cada artefacto;
- que metricas se obtuvieron;
- que limitaciones permanecen.

Capítulo 4

Metodologia

4.1. Caso de uso industrial

El caso de uso seleccionado es la detección de anomalías en rodamientos de motores eléctricos mediante señales de vibración. La elección permite trabajar con datos industriales públicos, etiquetas interpretables y un flujo temporal adecuado para ventanas, features y tensores.

4.2. Pipeline de datos

El flujo metodológico para el MVP queda definido como:

```
datos .mat crudos
-> manifiesto del dataset
-> extracción de señales
-> normalización de frecuencia de muestreo
-> perfilado estadístico
-> limpieza determinista
-> ventanas temporales
-> features y tensores
-> particiones train/validation/test
-> modelos de anomalías
-> evaluación
-> informe técnico
```

4.3. Manifiesto del dataset

El primer artefacto metodológico será un manifiesto con una fila por fichero crudo. Este manifiesto conectará los ‘.mat’ originales con el resto del pipeline. Las columnas previstas son:

Campo	Descripción
file_id	identificador del fichero original
dataset	nombre del dataset
source_path	ruta del fichero crudo
label	clase ‘normal’ o ‘fault’

fault_type	tipo de fallo si existe
fault_diameter_inch	diametro del defecto
load_hp	carga del motor
rpm	regimen de giro
sensor_channel	canal de sensor usado
source_sample_rate_hz	frecuencia de muestreo original
target_sample_rate_hz	frecuencia objetivo del MVP
source_format	formato original
notes	observaciones

4.4. Extraccion y perfilado

El canal principal inicial sera ‘DE_time’, asociado al acelerometro del lado drive-end. El perfilado generara resúmenes estadísticos por fichero y por etiqueta, incluyendo número de muestras, canales detectados, valores no finitos, media, desviación típica, RMS, curtosis, factor de cresta y energía.

4.5. Estructuración temporal

La frecuencia objetivo inicial sera 12 kHz. Las ventanas candidatas seran de 2048 o 4096 muestras con solapamiento del 50 %. Esta decisión permite comparar features tabulares con modelos clásicos y conservar ventanas crudas para autoencoders posteriores.

4.6. Particiones

Para evitar fuga de información entre ventanas, no se usara un split aleatorio ingenuo. La estrategia inicial sera:

- entrenamiento con ventanas normales;
- validación con ventanas normales retenidas;
- test con ventanas normales retenidas y ventanas con fallo.

4.7. Metricas

Las métricas iniciales seran precision, recall, F1-score, ROC-AUC, PR-AUC, tasa de falsos positivos y matriz de confusión. Desde el punto de vista industrial, el criterio principal sera mantener alta sensibilidad ante fallos sin disparar en exceso los falsos positivos.

Capítulo 5

Implementacion

5.1. Estructura del repositorio

La implementacion se organiza en tres areas:

- ‘codigo/’: aplicacion, tests, datos, modelos, informes y documentacion tecnica;
- ‘memoria/’: documento LaTeX del TFM;
- ‘recursos/’: PDFs, referencias y diagramas auxiliares.

5.2. Estado actual

Hasta este punto se ha creado la estructura inicial de directorios, se han descargado los datasets crudos, se ha definido el flujo de datos del MVP y se han implementado el contrato inicial del estado global del grafo y los esquemas Pydantic de entradas, decisiones y resultados. Los datos crudos se conservan en ‘codigo/data/raw/’, junto con un README de trazabilidad.

5.3. Estado global implementado

El estado global se ha dividido en dos capas:

- ‘codigo/app/schemas/state.py’: modelos Pydantic estrictos para validar el estado, configuraciones, metricas, errores y artefactos;
- ‘codigo/app/graph/state.py’: ‘TypedDict’ usado por LangGraph, validador de estado e inicializador del caso CWRU.

El inicializador ‘create_initial_cwru_state’ crea una ejecucion preparada para generar el manifiesto del dataset. El estado resultante no contiene senales ni arrays, solo rutas, contexto experimental y campos reservados para artefactos futuros.

5.4. Contratos Pydantic implementados

Se han anadido tres grupos de contratos:

- ‘codigo/app/schemas/dataset.py’: manifiesto del dataset, canales y metadatos de fallo;
- ‘codigo/app/schemas/agent_decisions.py’: decisiones estructuradas del supervisor, limpiador, estructurador, modelador, evaluador y redactor;
- ‘codigo/app/schemas/executor_results.py’: resultados comunes y especializados de ejecutores deterministas.

Estos contratos consolidan la regla arquitectonica de que los agentes producen configuraciones o decisiones validables, mientras que los ejecutores devuelven rutas, metricas, errores y referencias a artefactos, pero no datos pesados.

5.5. Artefactos previstos

- ‘codigo/data/interim/cwru_bearing/manifest.csv’;
- ‘codigo/data/interim/cwru_bearing/profile.json’;
- ‘codigo/data/processed/cwru_bearing/clean_signals/’;
- ‘codigo/data/tensors/cwru_bearing/windows_features.csv’;
- ‘codigo/data/tensors/cwru_bearing/windows_raw.npz’;
- ‘codigo/data/tensors/cwru_bearing/splits.json’;
- ‘codigo/reports/cwru_bearing/’;
- ‘codigo/models/cwru_bearing/’.

5.6. Ejecutor de manifiesto

Se ha implementado el primer ejecutor determinista del pipeline, el modulo ‘dataset_manifest.py’. Su responsabilidad es generar el artefacto:

```
codigo/data/interim/cwru_bearing/manifest.csv
```

a partir de los ficheros ‘.mat’ crudos de CWRU.

El ejecutor no carga senales completas. Solo inspecciona los nombres de fichero y aplica un mapeo controlado de metadatos CWRU: carga, RPM, tipo de fallo, diametro y posicion del fallo exterior cuando corresponde. El manifiesto generado conserva la frecuencia original de cada fichero y fija una frecuencia objetivo de 12 kHz para el MVP.

La ejecucion real sobre los datos descargados ha generado 64 registros: 4 ficheros normales y 60 ficheros con fallo. El fichero resultante contiene 65 lineas incluyendo la cabecera.

5.7. Dependencias

Se ha creado ‘requirements.txt’ como punto inicial para construir el entorno de Miniconda ‘tfm_v2’, fijado sobre Python 3.11. Incluye dependencias de contratos y grafo, procesamiento de datos, modelos clasicos, visualizacion tecnica y verificacion.

5.8. Verificacion actual

La verificacion de los contratos se ha realizado con tests unitarios en el entorno objetivo:

```
conda run -n tfm_v2 python -m unittest discover codigo/tests
```

Las pruebas validan la creacion del estado inicial CWRU, serializacion JSON, rechazo de campos no declarados, restricciones de configuraciones y metricas, coherencia del manifiesto, decisiones de agentes, resultados de ejecutores y generacion del manifiesto CWRU. En esta fase se han ejecutado 27 tests correctamente.

5.9. Siguiente paso de implementacion

El siguiente avance tecnico sera implementar el perfilador de datos, que leera el manifiesto generado, inspeccionara los canales reales de cada fichero ‘.mat’ y producira ‘codigo/data/interim/cwru_bearing/profile.json’.

Capítulo 6

Experimentos

6.1. Protocolo experimental

Esta seccion documentara los experimentos ejecutados sobre CWRU Bearing Dataset. Cada experimento debera indicar:

- version del manifiesto;
- configuracion de ventanas;
- features o tensores usados;
- modelo e hiperparametros;
- estrategia de particion;
- metricas obtenidas;
- artefactos generados.

6.2. Experimento base

El primer experimento comparara modelos clasicos de deteccion de anomalias entrenados con ventanas normales y evaluados contra ventanas normales retenidas y ventanas con fallo.

6.3. Experimentos posteriores

Experimentos previstos:

- sensibilidad al tamano de ventana;
- comparacion entre features temporales y frecuenciales;
- validacion leave-one-load-out;
- validacion sobre NASA IMS cuando el MVP este estabilizado.

Capítulo 7

Resultados

7.1. Resultados cuantitativos

Esta seccion recogera las metricas de cada experimento.

Modelo	Precision	Recall	F1	ROC-AUC	PR-AUC
Pendiente	–	–	–	–	–

Tabla 7.1: Plantilla de resultados para modelos de deteccion de anomalias.

7.2. Analisis cualitativo

Esta seccion interpretara falsos positivos, falsos negativos, sensibilidad a la carga del motor, robustez por tipo de fallo y trazabilidad de artefactos.

7.3. Limitaciones observadas

Cada resultado debera acompanarse de limitaciones, supuestos y riesgos de generalizacion industrial.

Capítulo 8

Conclusiones

8.1. Conclusiones principales

Esta seccion resumira las aportaciones del proyecto, separando los resultados del pipeline, la arquitectura multiagente y los modelos de deteccion de anomalias.

8.2. Trabajo futuro

Lineas previstas:

- ampliar la validacion con NASA IMS Bearing Dataset;
- incorporar autoencoders para ventanas crudas;
- anadir persistencia de checkpoints;
- exponer el MVP mediante API;
- preparar ejecucion Docker y SLURM cuando el flujo local sea estable.

Bibliografía

- [1] Case Western Reserve University Bearing Data Center. 12k drive end bearing fault data, 2026. Disponible en: <https://engineering.case.edu/bearingdatacenter/12k-drive-end-bearing-fault-data>.
- [2] Case Western Reserve University Bearing Data Center. Bearing data center, 2026. Disponible en: <https://engineering.case.edu/bearingdatacenter/welcome>.
- [3] Case Western Reserve University Bearing Data Center. Normal baseline data, 2026. Disponible en: <https://engineering.case.edu/bearingdatacenter/normal-baseline-data>.
- [4] NASA Prognostics Center of Excellence. Data set repository, 2026. Disponible en: <https://www.nasa.gov/intelligent-systems-division/discovery-and-systems-health/pcoe/pcoe-data-set-repository/>.
- [5] NASA Prognostics Center of Excellence. Ims bearings dataset, 2026. Disponible en: <https://phm-datasets.s3.amazonaws.com/NASA/4.+Bearings.zip>.